Performance Evaluation of Managed Kubernetes and Database-as-a-Service on AWS and Microsoft Azure

Dimitar Rashko
Faculty of Computer Science and Engineering
Skopje, Macedonia
dimitar.rashko@students.finki.ukim.mk

***Abstract*—This paper evaluates an end-to-end cloud workload deployed on managed Kubernetes and managed PostgreSQL services from Amazon Web Services (AWS) and Microsoft Azure. The same containerized FastAPI application, Kubernetes resource limits, autoscaling policy, and Python workload generator were used on both platforms. Experiments compared a single application pod with Kubernetes Horizontal Pod Autoscaler (HPA) operation under request rates from 1 to 1000 requests per second. The evaluation focuses on execution time, successful throughput, latency speedup, scalability, and elasticity. Results show that HPA substantially improves performance at moderate loads, especially around 100–200 requests per second, while both deployments eventually saturate as offered load exceeds the capacity of the fixed two-node clusters and the database-backed application path.**

I. Introduction

Cloud platforms increasingly provide managed infrastructure services that reduce the operational effort required to deploy scalable applications. Two important examples are managed Kubernetes, which provides container orchestration, and Database-as-a-Service (DBaaS), which provides managed database provisioning, maintenance, and availability. Although these services simplify deployment, their behavior under increasing and rapidly changing load still depends on resource limits, autoscaling, application design, and database capacity.

This work compares an equivalent application stack on AWS and Microsoft Azure. AWS Elastic Kubernetes Service (EKS) was combined with Amazon RDS for PostgreSQL, while Azure Kubernetes Service (AKS) was combined with Azure Database for PostgreSQL Flexible Server. The same application image performs CPU work for at least five seconds and writes a result to PostgreSQL before returning a response. A custom asynchronous workload generator applies controlled request rates to both environments. Single-pod and HPA-enabled experiments are used to evaluate execution time, throughput, speedup, scalability, and elasticity while keeping the Kubernetes application configuration consistent across providers.

II. Problem Description

The objective is to evaluate managed Kubernetes and DBaaS. Each request enters through a cloud load balancer, reaches a Kubernetes pod, performs application processing, inserts a row into the managed PostgreSQL database, and returns only after the database operation succeeds.

The AWS deployment used EKS in `eu-central-1` with two `t3.medium` worker nodes and Amazon RDS PostgreSQL on a `db.t3.micro` instance with 20 GB storage. The Azure deployment used AKS in `polandcentral` with two `Standard_B2ls_v2` worker nodes and Azure Database for PostgreSQL Flexible Server using the Burstable `Standard_B1ms` SKU. Images were stored in Amazon ECR and Azure Container Registry respectively. Both Kubernetes deployments exposed the application with a `LoadBalancer` service.

The application pod requested 250 mCPU and 128 MiB memory and was limited to 500 mCPU and 256 MiB memory on both clouds. HPA used CPU utilization with a 60% target, a minimum of one pod, a maximum of ten pods, and a 120-second scale-down stabilization window. Worker-node autoscaling was not enabled, so only the number of application pods changed during the HPA tests.

III. Workload Generator

The workload generator was implemented in Python using `asyncio` and `aiohttp`. For each test, a target number of requests per second (RPS) and a test duration were specified. The generator then sent requests at approximately the selected rate throughout the test. For example, a test configured for 100 RPS over 60 seconds would attempt to send a total of 6,000 requests.

The `/work` endpoint performs 10,000 SHA-256 rounds, waits if necessary so that processing lasts at least five seconds, and then inserts one result row into PostgreSQL. Final load tests used 60-second request-generation intervals at 1, 10, 50, 100, 200, 300, 400, 500, 600, 700, 800, 900, and 1000 target requests per second (RPS). Separate runs were performed with a single pod and with HPA enabled. The generator records successful and failed requests, success rate, average latency, and successful throughput. Throughput is calculated as successful responses divided by total wall-clock time. Latency speedup is defined as

$$S(r) = \frac{T_{\text{single}}(r)}{T_{\text{HPA}}(r)}, \tag{1}$$

where values above 1 indicate lower latency with HPA.

TABLE I
EXPERIMENTAL CLOUD CONFIGURATION

Provider	Kubernetes	Workers	Managed PostgreSQL	Region	Registry
AWS	EKS	2 × t3.medium	RDS db.t3.micro, 20 GB	eu-central-1	ECR
Azure	AKS	2 × B2ls_v2	Flexible Server B1ms	polandcentral	ACR

IV. RESULTS

A. Execution Time

At low load, both providers remained close to the workload minimum of five seconds. HPA became most beneficial when the single pod began to saturate. At 100 RPS, AWS average latency decreased from 45.41 s with one pod to 6.13 s with HPA, while Azure decreased from 14.14 s to 5.39 s. At 200 RPS, HPA latency was 7.17 s on AWS and 5.80 s on Azure. Above approximately 300 RPS, high failure rates and queueing caused latency to fluctuate rather than increase monotonically. The zero AWS single-pod latency at 800 RPS represents zero successful responses, not faster execution.

B. Throughput and Scalability

The single-pod deployments handled the 1–50 RPS range reliably but saturated as load increased. HPA substantially extended useful capacity. At 200 RPS, successful throughput reached 178.31 RPS on AWS and 171.27 RPS on Azure, compared with 33.39 and 32.66 RPS for the corresponding single-pod tests. At 300 RPS, HPA still achieved 93.78 RPS on AWS and 143.64 RPS on Azure, but success rates fell to 79.79% and 64.71%. At still higher loads, achieved throughput remained far below the requested rate, showing that adding pods alone could not remove the combined cluster and database bottlenecks.

C. Speedup

The strongest latency speedup occurred near the onset of single-pod saturation. AWS reached approximately 7.41× speedup at 100 RPS and 5.36× at 200 RPS. Azure reached approximately 2.62× at 100 RPS and 3.80× at 200 RPS. At heavier loads the speedup sometimes dropped below 1 because HPA reacted after load increased and both systems were already operating with substantial request failures. Therefore, high-load latency speedup must be interpreted together with success rate and throughput because average latency is calculated only from successful requests.

D. Elasticity

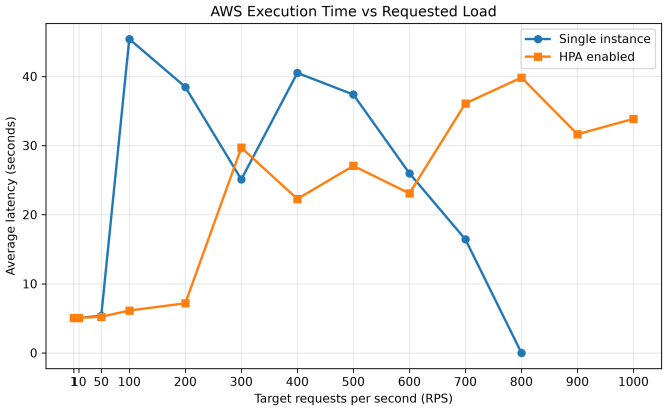
Elasticity was tested with a low–high–low sequence of 10, 100, and 10 RPS while HPA remained enabled. AWS throughput changed from 8.58 to 73.88 and then returned to 9.73 RPS. Azure changed from 9.24 to 67.62 and returned to 9.24 RPS. Both systems therefore recovered their low-load throughput after the burst. The Azure post-burst observation showed three replicas still running at only 5% CPU utilization, which is consistent with the configured 120-second scale-down stabilization window.

E. Cost Considerations

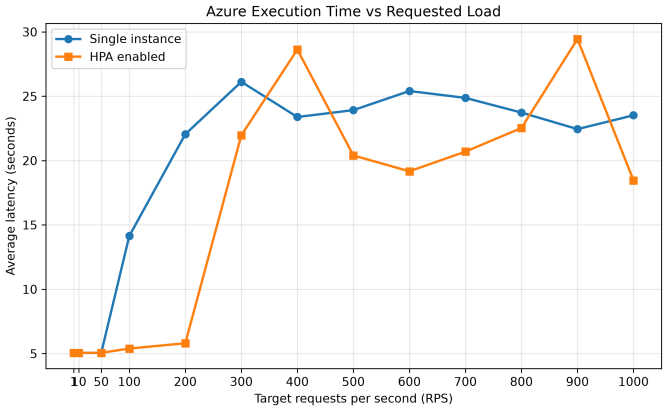
The experiment kept two worker nodes active on each provider and used HPA only for pod scaling. Additional cost components included the managed PostgreSQL instance, load balancer, container registry, storage, and provider-specific managed-service charges. Resources were stopped when testing was paused and deleted after completion to minimize unnecessary consumption. The cost from Microsoft Azure was $1.01, and from AWS it was $1.56.

V. CONCLUSION

The experiments show that HPA improves the ability of both managed Kubernetes deployments to absorb moderate increases in load, with the clearest benefits around 100–200 RPS. However, pod autoscaling alone cannot provide linear scalability when the underlying cluster remains fixed and every request also depends on a managed database operation. At high offered rates, throughput plateaus, success rates fall, and latency becomes less representative because only successful requests contribute to the average. The results demonstrate the importance of evaluating Kubernetes and DBaaS as a complete application path and of considering throughput, latency, failures, and autoscaling behavior together when comparing cloud platforms.

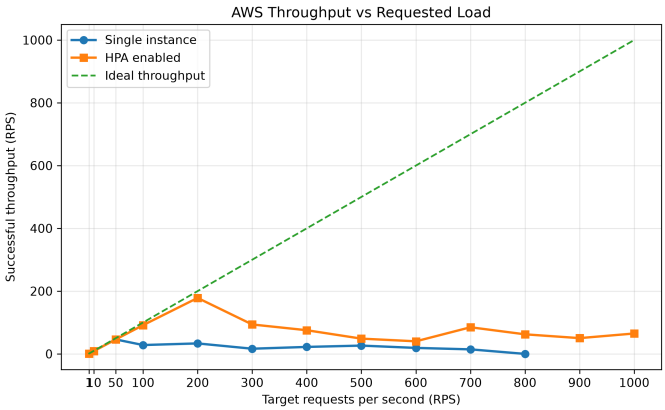


(a) AWS

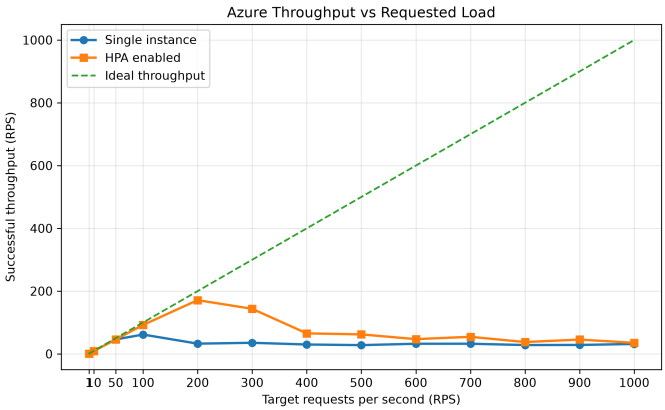


(b) Azure

Fig. 1. Average latency under single-pod and HPA-enabled execution.

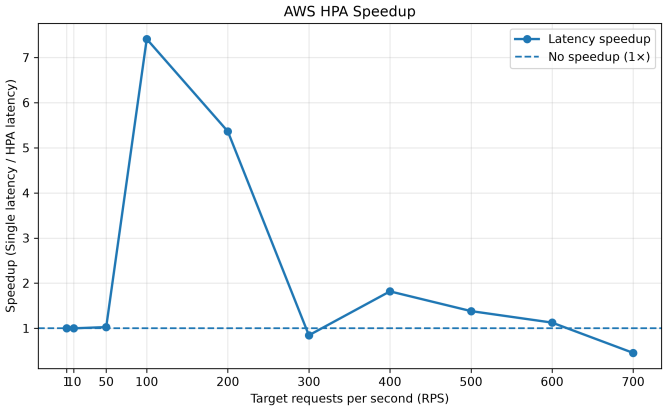


(a) AWS

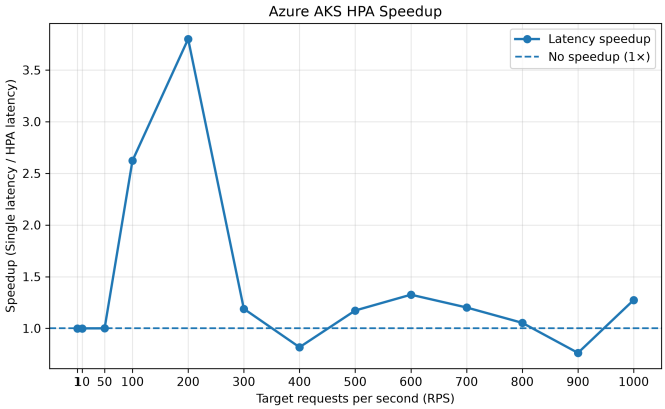


(b) Azure

Fig. 2. Successful throughput compared with requested load.

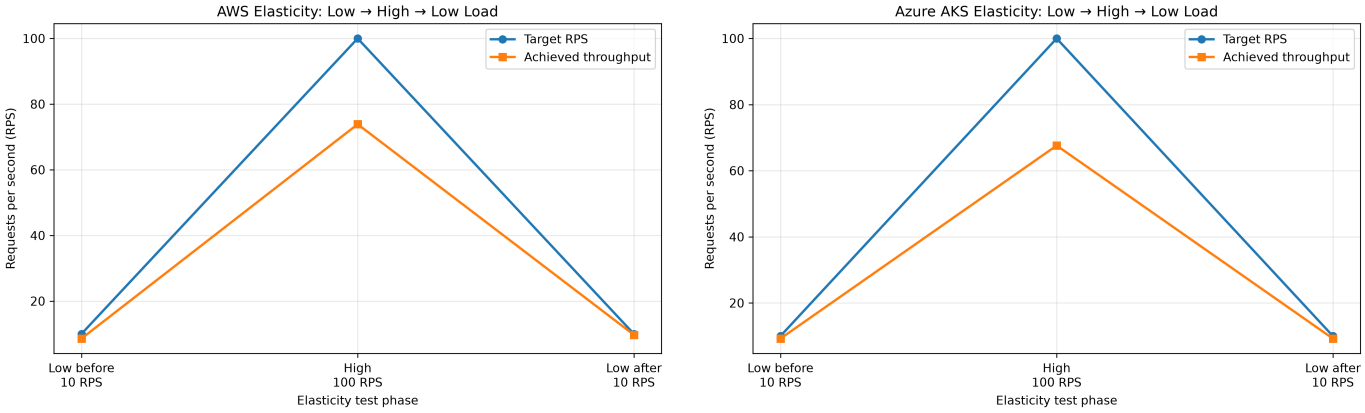


(a) AWS



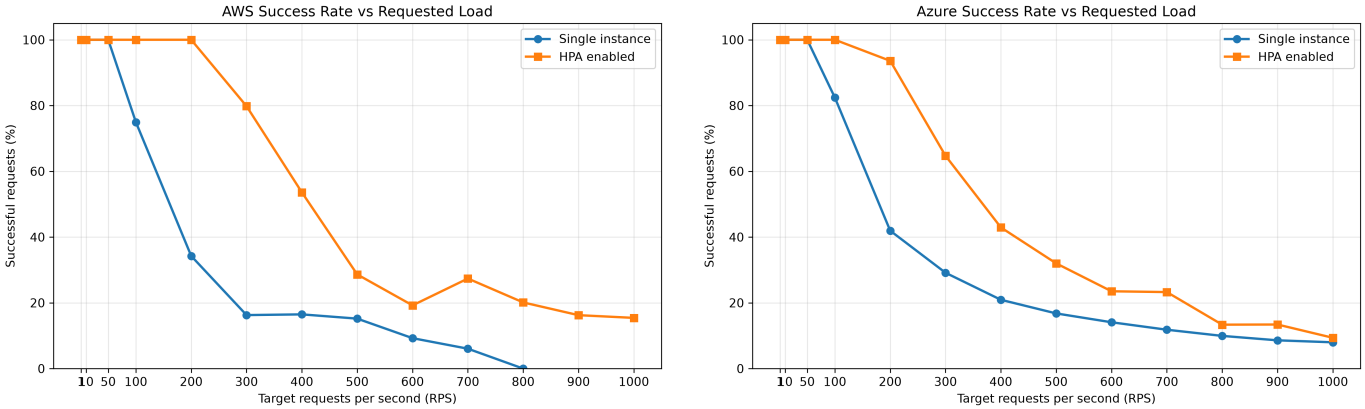
(b) Azure

Fig. 3. Latency speedup obtained from HPA relative to a single pod.



(a) AWS (b) Azure

Fig. 4. Elasticity comparison between AWS and Azure.



(a) AWS (b) Azure

Fig. 5. The success rate relative to the requested load.